Autenticação, autorização e JWT

Como funciona a implementação atual do AprovaAí

IDENTIDADE

Do formulário à decisão de acesso

Um guia para entender o cadastro, a confirmação de e-mail, o login, as sessões, a renovação de tokens e as regras de permissão do projeto.

FastAPI PostgreSQL Ed25519 Next.js

O que este documento descreve

O comportamento observado nos arquivos do workspace em **6 de novembro de 2026**. Os exemplos usam dados fictícios. Credenciais reais, valores do `.env`, chaves privadas e tokens de usuários não foram incluídos.

O projeto chama essa área de serviço de identidade. Hoje ela está implementada como módulos de uma mesma API, com persistência compartilhada. Ao longo da leitura, “serviço” significa a classe responsável por uma regra de negócio, e não necessariamente um microsserviço implantado de forma independente.

Edição de leitura • 1.0

22 páginas • fundamentos, fluxos, referência de rotas e operação

Base de referência: revisão 4ba38ee9 e alterações locais do workspace.

Roteiro de leitura

Comece pelo fluxo completo; depois consulte os detalhes do token.

Arquitetura: onde cada responsabilidade fica 03

Banco, sessões de banco e entidades 04

Cadastro, senhas e confirmação de e-mail 05

Login e criação de uma sessão 06

JWT: estrutura, assinatura e leitura 07

Campos do JWT emitido pelo projeto 08

Como uma requisição é autenticada 09

Autorização por papel e por proprietário 10

Refresh token: renovação e reutilização 11

Logout e invalidação imediata de acesso 12

Recuperação e redefinição de senha 13

Fila de e-mail e worker 14

Frontend, cookies e restauração de sessão 15

Swagger e acesso administrativo 16

Referência: rotas de autenticação e administração 17

Referência: perfis, documentos e notificações 18

Configuração e execução local 19

Proteções, limites e respostas HTTP 20

Testes, diagnóstico e limites da implementação 21

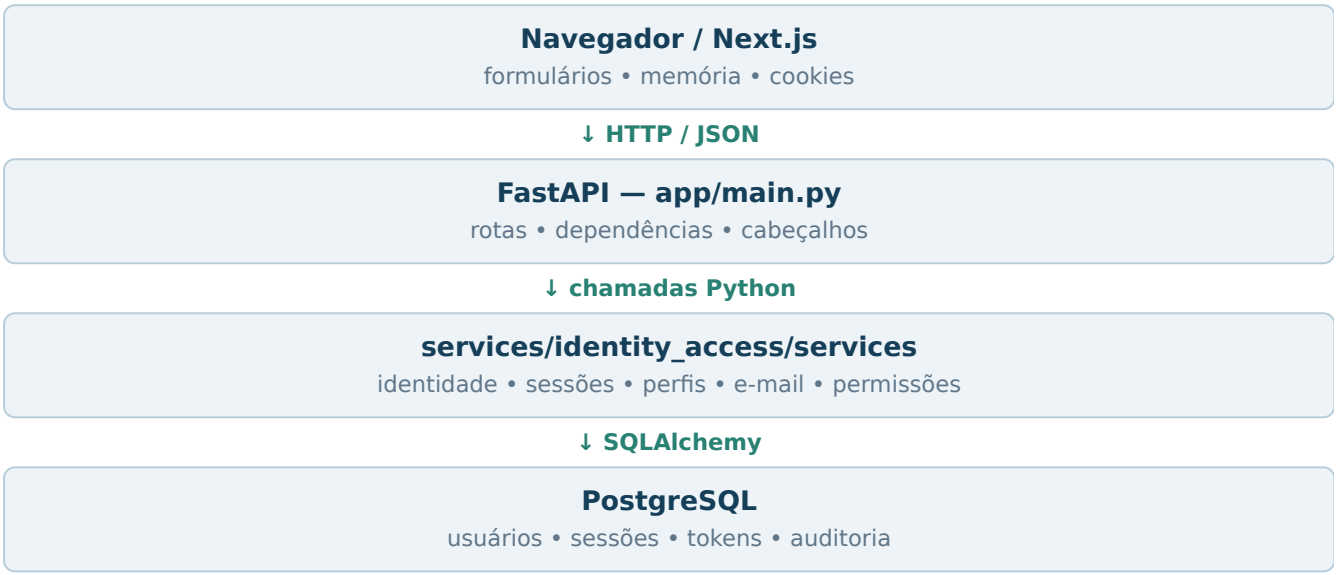
Glossário, código-fonte e referências 22

> **Duas perguntas diferentes**
> Autenticação responde “quem está fazendo a requisição?”. Autorização responde “essa pessoa pode executar esta ação sobre este recurso?”. Um login bem-sucedido não concede acesso automático a todas as rotas.

Os tempos apresentados são os valores padrão de `Settings`. Variáveis de ambiente podem alterá-los. A documentação técnica anterior em LaTeX contém propostas mais amplas; este guia se limita ao comportamento efetivamente implementado.

1. Arquitetura atual

Uma API, serviços de negócio e um worker que compartilha a persistência.



Como o código se distribui

`main.py` cria o aplicativo, configura o CORS e registra as rotas. A função `create_router()` concentra as rotas transferidas de `services/identity_access/api.py`, usando o prefixo `/api/v1`. O arquivo `api.py` mantém dependências e auxiliares HTTP.

`IdentityService` trata cadastro, credenciais, confirmação, recuperação, papéis e status. `SessionService` cria e valida sessões, renova tokens e revoga acessos. Os demais serviços cuidam de perfis, arquivos, notificações, candidaturas e limitação de tentativas.

O worker é um processo separado

`email_worker.py` lê a fila no mesmo banco e chama Brevo ou SMTP. Separar esse processamento evita fazer o usuário esperar pela entrega de e-mail na requisição de cadastro.

> **Por que não são microsserviços independentes?**
> As classes são chamadas diretamente, usam o mesmo código e recebem sessões do mesmo banco. Ainda não existe uma API de identidade implantada separadamente de uma API de cursos, nem banco autônomo para cada classe. Uma futura separação exigiria contratos de comunicação e uma estratégia de identidade entre aplicações.

Base no código: [C1], [C2], [C3], [C4], [C7]

2. Banco e modelo de dados

Uma configuração de conexão; várias sessões de trabalho e sessões de usuário.

`database.py` lê `DATABASE_URL` e cria o `engine` do SQLAlchemy com `pool_pre_ping=True`. Esse engine administra o acesso às conexões. `SessionLocal` fabrica unidades de trabalho; `get_session()` fornece uma sessão para a requisição e a fecha ao terminar.

Isso não significa uma única conexão física usada por todo o sistema. O pool atende às necessidades de cada processo. API e worker podem ter seus próprios engines e pools, apontando para o mesmo banco configurado.

Entidade / tabela	O que representa
`User / aa_users`	Identidade, hash da senha, papel, status, confirmação e `token_version`.
`AuthSession / aa_sessions`	Uma sessão de login: usuário, criação, expiração, atividade e revogação.
`RefreshToken / aa_refresh_tokens`	Hash do refresh, família, consumo, sucessor, expiração e revogação.
`EmailToken / aa_email_tokens`	Confirmação de e-mail: hash, propósito, validade e consumo.
`PasswordReset / aa_password_resets`	Código de redefinição: hash, validade e uso único.
`EmailOutbox / aa_email_outbox`	Mensagem pendente, token cifrado, tentativas e resultado de entrega.
`AuditEvent / aa_audit_events`	Registro de eventos relevantes, atores, resultado e correlação.
`RateLimitBucket`	Contadores persistentes de tentativas por janela.
`Profile, MediaItem, TeacherApplication, Notification`	Perfil, metadados dos arquivos, candidatura de professor e avisos privados.

Não confunda os dois tipos de sessão

Sessão SQLAlchemy: objeto temporário para ler e escrever no banco. **Sessão de autenticação:** linha em `aa_sessions`, que sobrevive a requisições e sustenta a validade de um login. Fechar a primeira não faz logout da segunda.

As rotas decidem quando executar `commit()`. Um `flush()` envia alterações ao banco dentro da transação, mas não substitui o commit. O fechamento de uma sessão com alterações não confirmadas permite que sejam desfeitas.

Base no código: [C1] database.py; [C5] models.py; [C4] SessionService

3. Cadastro e confirmação

A conta nasce como aluno e só pode entrar depois de confirmar o e-mail.

Passo a passo do cadastro

1. O frontend envia nome, e-mail e senha para `POST /api/v1/auth/register`.
2. O esquema `RegisterIn` valida os dados. O e-mail é normalizado; campos extras são rejeitados.
3. A rota aplica o limite de tentativas e verifica a disponibilidade de envio de e-mail.
4. `IdentityService.register()` verifica duplicidade e cria a conta com papel `student`, `is_active=False` e status `pending_verification`.
5. São criados o perfil, um token de confirmação e uma mensagem na fila de e-mail. A rota grava um evento de auditoria e confirma a transação.

Como a senha é armazenada

`security.py` usa `argon2.PasswordHasher` para produzir o hash da senha. A verificação compara a senha informada com esse hash; não recupera uma senha original. A biblioteca incorpora os parâmetros e o salt no formato armazenado.

Nos esquemas de cadastro e redefinição, a senha deve ter de **12 a 128 caracteres**, com pelo menos uma letra e um número. O cliente não escolhe o papel: tentar enviar `role="admin"` no cadastro é rejeitado pelo esquema.

Confirmação de uso único

O código de confirmação é um segredo aleatório, não um JWT de acesso. O banco guarda seu SHA-256. A validade padrão é de **24 horas**. Ao apresentar o token em `/auth/verify-email`, o serviço verifica hash, propósito, expiração e consumo; em seguida, marca o token como consumido e ativa o usuário.

Um novo token de confirmação invalida os anteriores ainda não consumidos. O reenvio responde de forma genérica, inclusive quando não existe uma conta pendente para o endereço informado.

> **Resultado prático**
> O cadastro retorna 201 e cria a conta, mas não autentica o usuário. O login antes da confirmação retorna 401. Um e-mail duplicado resulta em 409; um corpo inválido, em 422.

Base no código: [C2] register(), verify_email(), resend_verification(); [C3] IdentityService; [C6] schemas.py; [C8] security.py

4. Login e criação da sessão

A senha identifica o usuário; a sessão permite continuar autenticado.

1. O navegador envia e-mail e senha a `POST /api/v1/auth/login`.
2. A rota limita tentativas por IP e identidade. O serviço procura o e-mail normalizado e exige uma conta ativa, com status `active`.
3. A senha é verificada contra o hash Argon2. Falhas retornam a mensagem genérica “Credenciais inválidas.”.
4. `SessionService.create()` grava uma nova sessão, emite um JWT de acesso e gera um refresh token aleatório.
5. A rota registra o resultado de sucesso na auditoria, faz commit e devolve o JWT no JSON. O refresh segue em um cookie HttpOnly.

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "aluno@exemplo.com",
  "password": "uma senha de exemplo 9"
}

// Resposta ilustrativa: os tokens foram omitidos.
{
  "access_token": "<JWT_ASSINADO>",
  "token_type": "bearer",
  "expires_in_seconds": 900,
  "user": {"id": "<UUID>", "email": "aluno@exemplo.com", "role": "student"}
}
```

Por que dois tokens?

O **access token** autoriza chamadas por um período curto: 15 minutos por padrão. O **refresh token** permite obter novos tokens de acesso, sem reenviar a senha, enquanto a sessão continuar válida. A validade inicial padrão do refresh e da sessão é de sete dias.

O JSON de login não contém o hash da senha nem o refresh token. A resposta inclui `Cache-Control: no-store`. O serviço permite mais de uma sessão para um mesmo usuário: dois logins podem representar dois dispositivos.

Base no código: [C2] login(); [C3] authenticate(); [C4] create(); [C6] TokenOut

5. JWT: o que ele é

Um conjunto de informações assinado; o conteúdo continua legível.

No projeto, o JSON Web Token é usado como **access token assinado**. Ele transporta a identidade do usuário, a sessão, tempos de validade e outros campos. O formato efetivamente emitido usa três partes separadas por pontos:



Parte	Conteúdo neste projeto
Cabeçalho	Metadados: algoritmo `EdDSA`, identificador de chave `kid` e tipo de token JWT.
Payload	Objeto JSON com declarações, também chamadas de *claims*: `sub`, `sid`, `exp` e outras.
Assinatura	Resultado criptográfico calculado a partir do cabeçalho e do payload codificados, usando a chave privada.

Codificar não é cifrar

Cabeçalho e payload são representados em Base64url. Isso permite transportá-los em texto, mas não esconde seu conteúdo. A implementação usa `jwt.encode()` para assinar e não cria um token cifrado JWE. Por esse motivo, o JWT emitido não carrega a senha do usuário nem outros segredos de configuração.

O que a assinatura protege

A assinatura permite detectar uma alteração não autorizada nos dados. Se alguém trocar o papel no payload sem conseguir assinar novamente com a chave privada válida, a verificação falha. A API nunca deve confiar apenas em um payload decodificado.

Mesmo uma assinatura correta não garante, sozinha, acesso ao recurso. O destinatário, o emissor, os tempos, o propósito e o estado atual da sessão também precisam ser aceitos.

JWT não substitui HTTPS

O cabeçalho `Authorization: Bearer <token>` apresenta o token à API. Quem obtiver um token utilizável pode tentar apresentá-lo como credencial. A assinatura protege sua integridade; o transporte seguro e o tratamento do armazenamento resolvem outros problemas.

A RFC 7519 define o formato JWT. Aqui estamos descrevendo a modalidade assinada efetivamente escolhida pelo código, não afirmando que todo JWT existente tem obrigatoriamente essas três partes.

Base no código: [C8] create_access_token(), decode_access_token(); referência conceitual [R1]

6. Os campos do nosso JWT

Como relacionar as claims ao usuário, à sessão e ao relógio.

Campo	Significado e uso na implementação
`sub`	Identifica o usuário. O serviço usa esse valor para consultar `User`.
`sid`	Identifica a sessão de autenticação consultada em `AuthSession`.
`role`	Papel no instante da emissão: ALUNO, PROFESSOR ou ADMIN. A autorização usa o usuário atual do banco.
`iss`	Emissor esperado. Padrão: `aprovaai-auth`.
`aud`	Destinatário esperado. Padrão: `aprovaai-api`.
`iat`	Instante de emissão, representado numericamente em segundos desde a época Unix.
`nbf`	Instante a partir do qual o token pode ser aceito; na emissão atual coincide com `iat`.
`exp`	Expiração. Por padrão, 15 minutos depois da emissão.
`jti`	Identificador aleatório do JWT. É exigido, mas não há lista própria de revogação por jti.
`ver`	Versão do usuário na emissão. Deve coincidir com `User.token_version`.
`typ`	Claim do payload: `access` para a API ou `admin_docs` para a documentação.

Exemplo de metadados e conteúdo

```
// Cabeçalho ilustrativo
{"alg":"EdDSA","kid":"ed25519-1","typ":"JWT"}

// Parte do payload, com identificadores fictícios
{"sub":"<UUID_USUARIO>","sid":"<UUID_SESSAO>",
 "role":"ALUNO","iss":"aprovaai-auth","aud":"aprovaai-api",
 "typ":"access","ver":0,"jti":"<UUID_TOKEN>"}
```

O `typ` do cabeçalho e o `typ` do payload ficam em lugares distintos. O código separa os usos “acesso à API” e “documentação administrativa” por meio da claim no payload. As claims `sid`, `ver` e `role` fazem parte do contrato específico da aplicação.

Base no código: [C8] security.py; [C4] SessionService; [R1] para a terminologia geral

7. Validação de uma requisição

A assinatura é a primeira barreira; o banco decide se o login continua válido.

Etapa A — extrair a credencial

`AuthDependencies.user()` usa `HTTPBearer(auto_error=False)`. A dependência lê o cabeçalho Authorization; na ausência de credenciais aceitas, responde 401. Em seguida chama `SessionService.authenticate()`.

Etapa B — validar o JWT

1. O cabeçalho é lido para verificar o algoritmo e o `kid`. Essa leitura não autentica o token.
2. O algoritmo permitido é fixado em `EdDSA`; o identificador deve ser o configurado em `AA_JWT_KID`.
3. O PyJWT verifica a assinatura com a chave pública, o emissor, o destinatário e os campos temporais aplicáveis.
4. O código exige a presença de `exp, iat, nbf, sub, jti, sid, iss, aud, typ, ver`, confere o propósito e verifica que sub, sid e jti sejam strings não vazias.

Etapa C — conferir usuário e sessão

O serviço consulta a sessão por `sid` e o usuário por `sub`. Ambos devem existir. A sessão precisa pertencer ao usuário, não estar revogada e não estar expirada. A conta deve estar ativa, com status `active`, e a versão do token deve coincidir com a versão atual do usuário.

> **Exemplo: token ainda dentro dos 15 minutos**
> Se o administrador suspender a conta, o JWT pode continuar com assinatura e expiração corretas. Mesmo assim, a requisição seguinte será recusada porque a sessão e/ou o estado do usuário no banco já mudou.

Como funcionam as chaves

A chave privada Ed25519 assina; a chave pública verifica. A configuração aceita chaves PEM codificadas em Base64url e confere se o par corresponde. Se a chave pública não for fornecida, ela é derivada da privada. Base64 é apenas a representação do arquivo, não uma proteção do segredo.

Há um par e um `kid` configurados por processo. Não há busca dinâmica de chaves em JWKS nem uma janela automática de aceitação de várias chaves. As boas práticas de validação são discutidas na RFC 8725; a integração JOSE com Ed25519 consta na RFC 8037.

Base no código: [C7] AuthDependencies; [C4] authenticate(); [C8] decode_access_token(); [C9] config.py; [R2], [R3]

8. Autorização: quem pode fazer o quê

Papel atual no banco e relação de propriedade do recurso.

Depois da autenticação, a rota pode exigir um papel com `auth.roles(...)`. Essa dependência chama `IdentityService.require_roles()`, que exige conta ativa e compara `user.role` com os papéis permitidos.

Regra	Exemplos efetivamente implementados
Qualquer usuário autenticado	Consultar a própria identidade, perfil, sessões, notificações e candidatura.
Somente `student`	Enviar documento e criar candidatura para professor.
Somente `teacher`	Acessar a rota de teste `/auth/test/teacher`.
Somente `admin`	Alterar papel/status de usuários e acessar a documentação administrativa.
Proprietário ou administrador	Baixar um documento pelo serviço de mídia.
Somente proprietário	Revogar uma sessão via rota individual ou marcar a própria notificação como lida.

Administrador não é um passe universal

Os papéis não têm uma hierarquia implícita. Uma rota que exige exclusivamente professor não passa a aceitar administrador por consequência do nome do papel. O mesmo vale para envio de documentos, reservado ao aluno. Uma exceção administrativa precisa estar explícita na regra.

Propriedade é verificada no servidor

`require_owner()` compara o usuário com o dono do recurso e só aceita administrador quando `allow_admin=True`. As rotas de sessões e notificações também filtram os objetos pelo usuário. Conhecer o UUID de outro recurso não concede acesso.

No download de arquivo alheio, a rota retorna 403. Para uma sessão ou notificação fora do conjunto do usuário, as rotas retornam 404. São contratos diferentes de resposta, embora ambos impeçam o acesso indevido.

> **O papel do frontend**
> Menus e redirecionamentos ajudam a navegação. Eles não substituem as verificações da API. A aplicação não confia em um papel que o navegador envie em um formulário ou cookie para permitir uma operação protegida.

A candidatura de professor cria um pedido pendente e uma notificação. Esse fluxo, isoladamente, não transforma o aluno em professor. A alteração de papel passa pelo endpoint administrativo de identidade.

Base no código: [C7] roles(); [C3] require_roles(), require_owner(); [C2] rotas; [C10] serviços de mídia, notificações e candidatura

9. Refresh token e rotação

Renovar o login com um segredo de uso controlado e persistência no banco.

O refresh token não é JWT. `generate_secret()` produz 32 bytes aleatórios por meio de `secrets.token_urlsafe(32)`. A string é enviada ao navegador; o banco armazena seu SHA-256, e não o valor original. O segredo tem alta entropia, por isso seu uso difere do de uma senha humana protegida por Argon2.

O que acontece em /auth/refresh

1. A rota exige uma origem permitida, aplica o limite de tentativas e lê o cookie `aprovaai_refresh`.
2. O serviço calcula o hash, busca o registro e solicita bloqueio da linha no banco.
3. Verifica se o refresh já foi usado ou revogado. Confere também sua validade, a sessão e a conta.
4. Uma atualização condicional marca o token como usado. Se outra execução já o consumiu, o serviço trata isso como reutilização.
5. É criado um sucessor com novo segredo, mesma família e validade padrão de sete dias a partir da renovação. O predecessor aponta para o sucessor.
6. A expiração da sessão é atualizada. São emitidos novo JWT e novo cookie; a rota confirma a transação.

Refresh A	→ usado	**Refresh B**	→ usado	**Refresh C**	→ atual

O que é uma família?

Nesta implementação, a família usa o identificador da sessão. A sequência de refreshes pertence ao mesmo login. Se o token A reaparecer depois de ter sido consumido, o serviço revoga a sessão e seus refreshes, faz commit e devolve 401. A invalidação permanece gravada mesmo que a chamada tenha falhado.

> **A validade pode deslizar**
> Cada renovação bem-sucedida cria um novo prazo e atualiza o da sessão. Não há no código um teto absoluto de duração independente das renovações. Sete dias descrevem a validade padrão corrente, e não um encerramento obrigatório sete dias após o primeiro login.

Requisições concorrentes com o mesmo refresh merecem atenção: a aplicação tem bloqueio e atualização condicional, mas uma disputa pode revogar a família. O frontend compartilha a renovação em andamento dentro da mesma instância JavaScript; abas distintas ainda precisam ser avaliadas em testes reais.

Base no código: [C4] refresh(), revoke_session(); [C8] generate_secret(), token_digest(); [C11] apiClient.ts

10. Logout e revogação

Como o acesso pode terminar antes de o JWT expirar.

Logout da sessão atual

`POST /auth/logout` aceita um access token Bearer ou, quando ele não é informado, o cookie de refresh acompanhado de uma origem permitida. O serviço localiza a sessão e marca sua revogação. Os refreshes associados também recebem a marca de revogação.

A resposta remove os cookies de refresh e documentação, registra o evento de logout e informa que a sessão foi encerrada. Em chamadas posteriores, a checagem da sessão no banco rejeita o JWT antigo.

Encerrar uma sessão específica

`GET /auth/sessions` lista as sessões do usuário, incluindo os campos de revogação e expiração. A lista não é filtrada para mostrar apenas sessões ativas. A rota DELETE exige que o identificador pertença ao conjunto do próprio usuário; outro identificador resulta em 404.

Sair de todos os dispositivos

`POST /auth/logout-all` revoga as sessões ainda não revogadas do usuário e os refreshes vinculados. As sessões de outra pessoa não são afetadas. A resposta limpa os cookies pertinentes no navegador que fez a chamada.

Evento	Efeito implementado
Papel realmente alterado	Incrementa `token_version` e revoga sessões e refreshes ativos.
Status realmente alterado	Atualiza atividade/status, incrementa a versão e revoga sessões e refreshes ativos.
Senha redefinida	Atualiza o hash, incrementa a versão, invalida códigos de recuperação pendentes e revoga sessões/refreshes.
Mesmo papel novamente	Não incrementa a versão nem encerra sessões por mudança de papel.
Refresh reutilizado	Revoga a sessão/família correspondente; não precisa encerrar todas as sessões do usuário.

Por que usar token_version e sessão?

A sessão permite invalidar um login específico. A versão representa uma mudança global no estado de credenciais do usuário. Na validação, o serviço exige coerência entre esses controles e as claims do JWT.

Isso caracteriza um modelo com estado no servidor. O token pode ser verificado criptograficamente sem o banco, mas a implementação do projeto consulta a persistência para decidir se deve aceitá-lo.

Base no código: [C2] logout(), revoke_session(), logout_all(); [C4] revogações; [C3] set_role(), set_status(), reset_password()

11. Recuperação de acesso

Redefinição por código de uso único, sem expor a existência de uma conta.

Solicitar o e-mail

`POST /auth/forgot-password` e `/auth/password-recovery/request` executam a mesma função. A rota exige um endereço válido, verifica a disponibilidade do serviço de e-mail e aplica o limite de três tentativas por hora, por IP e identidade.

O serviço só produz um código para uma conta ativa com status `active`. Se a conta não existir ou não estiver apta, a resposta continua genérica: “Se o e-mail estiver cadastrado, você receberá instruções.”. A rota retorna 202 quando aceita o processamento.

Gerar e entregar o código

Os códigos anteriores ainda não usados são invalidados. Um novo segredo é criado, seu hash SHA-256 é salvo em `PasswordReset` e a validade padrão é de **20 minutos**. A fila recebe uma versão cifrada do segredo para que o worker consiga montar o e-mail.

O link produzido tem o formato `/recuperar-acesso#token=...`. O fragmento é lido pelo frontend e não é enviado automaticamente como parte do caminho HTTP. O formulário envia o código e a nova senha no corpo de uma requisição à API.

Confirmar a nova senha

1. As rotas `/auth/reset-password` e `/auth/password-recovery/confirm` validam a nova senha no esquema Pydantic.
2. O serviço procura o hash do código e recusa código ausente, expirado ou já usado.
3. Uma atualização condicional reivindica o uso do código para impedir o consumo repetido.
4. O hash da senha é substituído. A versão é incrementada, os códigos pendentes são invalidados e as sessões/refreshes do usuário são revogados.
5. A rota faz commit e pede que a pessoa entre novamente.

> **O que o usuário deve observar**
> Depois de redefinir, a senha anterior falha e a nova funciona. Uma sessão que estava aberta perde o acesso. O link utilizado não permite redefinir a senha uma segunda vez. Apenas receber o e-mail não altera a senha.

Os tokens de confirmação e recuperação são independentes dos access tokens. A finalidade de cada um é verificada pelo respectivo serviço e pela tabela correspondente.

Base no código: [C2] request_recovery(), confirm_recovery(); [C3] recuperação; [C6] PasswordResetIn; [C12] conteúdo dos e-mails

12. Fila de e-mail e worker

Enviar mensagens fora da requisição e persistir tentativas de entrega.

`EmailOutboxService.enqueue()` coloca a mensagem na mesma unidade de trabalho do cadastro ou da recuperação. Com isso, a aplicação pode confirmar os dados do fluxo e a intenção de enviar o e-mail na mesma transação do banco.

Por que há um token cifrado e outro em hash?

A tabela de validação precisa comparar um código apresentado com seu hash. O worker, por outro lado, precisa recuperar o código original para escrever o link. Por isso a outbox guarda `encrypted_token` cifrado com Fernet, usando `AA_EMAIL_ENCRYPTION_KEY`. São duas necessidades distintas.

Ciclo de processamento

1. O worker seleciona uma mensagem pendente cujo horário de tentativa já chegou. No PostgreSQL, usa bloqueio de linha com `skip_locked`.
2. Decifra o token, monta assunto e corpo e chama o remetente configurado.
3. Em sucesso, marca `delivered`, grava o horário e apaga o token cifrado da linha.
4. Em falha, incrementa as tentativas e registra um erro resumido com o tipo de exceção, sem persistir a mensagem bruta do provedor.
5. Agenda nova tentativa com espera exponencial, começando em 60 segundos. Ao chegar à oitava falha, marca `failed`.

Escolha do provedor

Se chave e remetente Brevo estiverem configurados, a implementação usa a API da Brevo. Caso contrário, escolhe SMTP. O cadastro e a recuperação consultam a disponibilidade de configuração e podem retornar 503 antes de gerar um fluxo incompleto.

```
cd backend
uv run --env-file .env python -m app.services.identity_access.email_worker
```

Quando não encontra trabalho, o loop espera dois segundos por padrão. O worker usa `SessionLocal`, portanto acessa o mesmo banco configurado para a aplicação.

> **Limite importante de entrega**
> Há rastreamento e novas tentativas, mas não uma garantia de envio exatamente uma vez. Se o provedor aceitar o e-mail e o processo falhar antes de confirmar o estado no banco, a mensagem pode ser reenviada. A coluna idempotency_key existe, mas não é enviada como mecanismo de deduplicação ao provedor no código atual.

Base no código: [C12] EmailOutboxService; [C13] NotificationService e provedores; email_worker.py

13. Frontend, cookies e sessão

O que fica em memória, o que fica no navegador e o que a API verifica.

`frontend/src/lib/apiClient.ts` usa `NEXT_PUBLIC_API_URL` para decidir se a integração está configurada. O JWT de acesso é mantido em uma variável do módulo JavaScript, não no localStorage. As requisições usam `credentials: "include"` para permitir o envio dos cookies aplicáveis.

Elemento	Comportamento observado
Access token	Vai no cabeçalho Authorization quando existe em memória. É perdido ao recarregar o contexto JavaScript.
`aprovaai_refresh`	Cookie HttpOnly, restrito ao caminho `/api/v1/auth`; SameSite e Secure vêm da configuração.
`aprovaai_admin_docs`	Cookie HttpOnly no caminho `/`, usado para as rotas de documentação.
Cookie de papel demonstrativo	Ajuda menus e navegação. Não autentica chamadas protegidas nem substitui o usuário do banco.

Como a sessão é restaurada

Ao receber 401 numa chamada protegida, o cliente tenta `/auth/refresh` e, se der certo, repete a chamada original uma vez com o novo JWT. Login, cadastro e demais operações públicas de autenticação ficam fora dessa renovação automática.

`ApiSessionSync` chama `/auth/me` quando a interface tem um papel anterior para validar. Esse acesso pode disparar o refresh. Em sucesso, atualiza as informações de navegação com o papel retornado pelo servidor; em 401, limpa o papel e redireciona ao login.

Proteções dos cookies

HttpOnly impede a leitura direta por JavaScript. **Secure** exige transporte HTTPS para o envio do cookie; em desenvolvimento HTTP a configuração costuma ser false. **SameSite** controla o envio em contextos entre sites; o padrão é lax. Essas opções não substituem autorização nem eliminam todos os riscos de execução de scripts indevidos.

Refresh e logout baseado em cookie exigem também um cabeçalho Origin aceito. O CORS permite credenciais para a origem do frontend configurada. Use o mesmo host nos endereços locais: misturar localhost e 127.0.0.1 pode mudar origem e comportamento dos cookies.

> **Escopo atual da integração**
> Login, cadastro, confirmação, recuperação, logout, identidade e notificações têm integração no cliente da API. Partes da interface continuam demonstrativas. Uma tela visual de professor ou administrador, isoladamente, não prova que a operação correspondente foi implementada no backend.

Base no código: [C11] apiClient.ts; ApiSessionSync.tsx; [C2] cookies/CORS; [C9] configurações

14. Swagger administrativo

Entrar na documentação e autorizar chamadas são duas etapas distintas.

Primeiro: abrir a documentação

A aplicação desabilita os caminhos automáticos de documentação e registra versões protegidas. Acesse `http://localhost:8000/admin/login`. O formulário exige uma conta ativa cujo papel atual seja `admin`.

Depois de conferir origem, limite e credenciais, a rota cria uma sessão e um JWT com propósito `admin_docs`, coloca esse JWT no cookie da documentação e redireciona para `/docs`. O schema `/openapi.json` exige o mesmo tipo de acesso.

Depois: executar endpoints protegidos

1. No Swagger, abra `POST /api/v1/auth/login` e faça login.
2. Copie o `access_token` retornado no JSON.
3. Clique em **Authorize** e informe o token no campo do esquema Bearer.
4. Use **Try it out** na rota desejada. A API continuará verificando permissões e propriedade.

A configuração é HTTP Bearer. Não há o fluxo OAuth2 password grant com endpoint `/auth/token`. O token da documentação e o da API não são intercambiáveis, mesmo que tenham sido emitidos para o mesmo administrador.

O problema de “Forbidden” no formulário

O login administrativo exige uma origem válida. A página do formulário agora usa `Referrer-Policy: same-origin`, preservando o contexto necessário ao POST de mesma origem no navegador. As demais respostas continuam usando no-referrer. A proteção não passou a aceitar Origin ausente, null ou arbitrário.

Criação inicial do administrador

```
cd backend
uv run --env-file .env python -m app.services.identity_access.bootstrap_admin admin@exemplo.com
```

O comando solicita nome e senha e só cria o administrador inicial se não existir nenhum. O cadastro normal não cria administradores. Para alterar papéis depois disso, a API dispõe de uma rota administrativa.

> **Diagnóstico rápido**
> Um 401 normalmente pede uma credencial válida. Um 403 pode significar papel insuficiente, origem não autorizada ou sessão administrativa não aceita. Leia o campo detail da resposta para distinguir os casos. Acesso sem sessão a /docs redireciona ao formulário.

Base no código: [C2] admin_docs_login(), _admin_docs_user(), admin_swagger_docs(), middleware; bootstrap_admin.py

15. Rotas de identidade

Os caminhos da tabela recebem o prefixo /api/v1.

Método e caminho	Acesso / resultado principal
`POST /auth/register`	Público; cadastro de aluno pendente. 201.
`POST /auth/login`	Público; valida credenciais e cria sessão. 200.
`POST /auth/verify-email`	Público; consome código válido e ativa a conta. 200.
`POST /auth/resend-verification`	Público; reenvio para conta pendente, resposta genérica. 202.
`POST /auth/refresh`	Cookie de refresh e origem permitida; rotaciona credenciais. 200.
`POST /auth/logout`	Bearer ou cookie com origem permitida; revoga a sessão. 200.
`GET /auth/sessions`	Bearer; lista sessões do próprio usuário. 200.
`DELETE /auth/sessions/{session_id}`	Bearer; revoga uma sessão do próprio usuário. 200.
`POST /auth/logout-all`	Bearer; revoga as sessões do próprio usuário. 200.
`GET /auth/me`	Bearer; retorna a identidade atual do banco. 200.
`POST /auth/forgot-password` `POST /auth/password-recovery/request`	Públicos; aliases para solicitar redefinição, resposta genérica. 202.
`POST /auth/reset-password` `POST /auth/password-recovery/confirm`	Públicos; aliases que exigem código válido e nova senha. 200.
`GET /auth/test/teacher`	Bearer e papel teacher; teste de autorização. 200.
`PATCH /admin/users/{user_id}/role`	Bearer e papel admin; altera papel. 200.
`PATCH /admin/users/{user_id}/status`	Bearer e papel admin; altera status. 200.

Público significa não exigir previamente uma sessão Bearer. Ainda há validação de dados, códigos de uso específico e, quando implementado na rota, limites de tentativas. O nome “público” não elimina essas verificações.

Ao ativar uma conta via alteração administrativa de status, o serviço exige `verified_at`. Não é possível usar essa rota para contornar a confirmação do e-mail de uma conta pendente. Os valores aceitos para status administrativo são active, suspended e disabled.

Base no código: [C2] create_router(); [C6] esquemas de entrada; [C3] regras de papel/status

16. Recursos relacionados

O mesmo mecanismo de identidade protege os recursos de cada usuário.

Método e caminho, com /api/v1	Regra aplicada
`GET /profiles/me`	Autenticado; consulta o próprio perfil.
`PATCH /profiles/me`	Autenticado; altera nome e preferências do próprio perfil.
`POST /media/documents`	Aluno; envia PDF, JPEG ou PNG compatível com a assinatura do arquivo.
`GET /media/documents/{document_id}`	Proprietário ou administrador; download como anexo.
`POST /teacher-applications`	Aluno; cria candidatura com documentos próprios e ainda não usados.
`GET /teacher-applications/me`	Autenticado; lê a própria candidatura.
`GET /notifications/me`	Autenticado; lista até 100 avisos do usuário.
`PATCH /notifications/{notification_id}/read`	Autenticado e proprietário; marca leitura de forma idempotente.

Perfil

O serviço acessa o perfil pela identidade já validada; não recebe um user_id livre no corpo para escolher outra conta. As preferências são limitadas a 30 entradas e os nomes passam pelo esquema de tamanho.

Documentos

O serviço limpa o nome original, cria um nome interno aleatório, verifica tamanho, extensão e assinatura inicial do conteúdo. O limite padrão é de 8 MiB. Os bytes ficam em `AA_MEDIA_ROOT`; o banco guarda metadados, dono e checksum. O download usa attachment, no-store e nosniff.

Uma falha de validação remove o arquivo parcial. Se o commit da rota falhar depois da gravação, a rota tenta desfazer a transação e remover o arquivo salvo. As verificações de assinatura inicial não equivalem a uma análise completa de malware.

Candidatura e notificações

A candidatura exige de um a cinco documentos distintos, todos existentes, próprios e não vinculados a outra candidatura. Uma conta só pode ter uma candidatura registrada. O pedido começa como pending e cria uma notificação privada.

Consultar ou marcar uma notificação de outra pessoa não é permitido, mesmo conhecendo o identificador. O endpoint de leitura devolve o mesmo horário quando a notificação já foi marcada.

Base no código: [C2] rotas; [C10] MediaService, ProfileService, TeacherApplicationService; [C13] notificações

17. Configuração e execução

Nomes de variáveis e valores padrão do código; nenhum segredo real.

Configuração	Função / padrão
DATABASE_URL	Conexão SQLAlchemy; necessária para inicializar o banco.
AA_JWT_PRIVATE_KEY_B64 AA_JWT_PUBLIC_KEY_B64	PEM em Base64url. A privada é exigida; a pública pode ser derivada.
AA_JWT_KID	Identificador de chave; padrão ed25519-1.
AA_JWT_ISSUER / AA_JWT_AUDIENCE	aprovaai-auth / aprovaai-api.
AA_ACCESS_TOKEN_MINUTES AA_REFRESH_TOKEN_DAYS	15 minutos / 7 dias.
AA_VERIFICATION_HOURS / AA_RESET_MINUTES	24 horas / 20 minutos.
AA_EMAIL_ENCRYPTION_KEY	Chave Fernet para recuperar tokens pendentes na fila.
AA_FRONTEND_ORIGIN / AA_PUBLIC_APP_URL	Origem permitida / endereço público para links.
AA_PASSWORD_RESET_URL	Página de redefinição; alinhar com /recuperar-acesso do frontend.
AA_COOKIE_SECURE / AA_COOKIE_SAMESITE	false / lax nos padrões locais; dependem do ambiente.
BREVO_API_KEY / BREVO_SENDER_EMAIL	Provedor Brevo. Alternativa: variáveis AA_SMTP_*.
NEXT_PUBLIC_API_URL	URL da API consumida pelo frontend; exemplo local localhost:8000.

Ordem de execução local

```
# Terminal da API, dentro de backend/
uv sync --locked
uv run --env-file .env alembic upgrade head
uv run --env-file .env uvicorn app.main:app --reload

# Outro terminal, dentro de backend/
uv run --env-file .env python -m app.services.identity_access.email_worker

# Terminal do frontend, dentro de frontend/
npm run dev
```

O código deve conseguir autenticar no PostgreSQL antes de criar usuários. As migrações criam as tabelas; não basta o banco existir. O arquivo .env do backend e o .env.local do frontend têm papéis diferentes.

A classe Settings tem um fallback histórico para a URL de redefinição em outra porta. Configure explicitamente AA_PASSWORD_RESET_URL para a página do frontend utilizada. Os valores efetivos do seu ambiente não foram lidos para produzir esta tabela.

Base no código: [C9] config.py; backend/.env.example; backend/README.md; [C11] frontend

18. Proteções e respostas HTTP

Como diferenciar erro de entrada, falta de credencial e falta de permissão.

Código	Significado no projeto / exemplo
400	Código de confirmação/recuperação inválido; documento incompatível.
401	Credencial ausente ou inválida; conta inativa; sessão expirada/revogada.
403	Origem rejeitada; papel insuficiente; acesso indevido a documento.
404	Recurso não encontrado ou fora do conjunto permitido em certas rotas.
409	E-mail duplicado; candidatura existente; ativação sem confirmação.
413	Documento excede o limite de tamanho.
422	Validação de entrada: senha curta, campos extras, papel inválido.
429	Limite de tentativas excedido. A resposta informa Retry-After.
503	Serviço indisponível, como configuração de envio de e-mail ausente.

Limites por IP e identidade

Operação	Tentativas por janela, em cada contador
Cadastro	5 em 600 segundos.
Login	10 em 900 segundos.
Confirmação / reenvio	10 em 3600 segundos / 3 em 3600 segundos.
Recuperação / refresh	3 em 3600 segundos / 30 em 60 segundos.
Login do Swagger	5 em 900 segundos.

O auxiliar consome os contadores de IP e identidade e faz commit antes de prosseguir. As chaves ficam em hash SHA-256; os contadores usam upsert no PostgreSQL ou SQLite. Isso persiste as tentativas mesmo quando o restante do login falha. Não há aplicação automática desse limitador em todas as rotas.

Auditoria e cabeçalhos

O middleware gera um X-Request-ID e inclui nosniff, X-Frame-Options: DENY e política de referrer. As rotas de identidade e documentação recebem no-store. Eventos de cadastro, confirmação, login, logout, mudança de papel/status e documentação têm registros explícitos; não há auditoria completa automática de toda leitura e escrita.

O CORS usa a origem configurada do frontend e permite credenciais. CORS é uma política de navegador; ele não substitui a validação de tokens ou o controle de permissão no backend.

Base no código: [C7] RateLimit e tradução de erros; [C14] RateLimitService; [C2] middleware e auditoria

19. Testes e diagnóstico

O que está verificado e o que ainda exige outros tipos de teste.

A suíte do backend

`backend/tests/test_authentication.py` reúne testes diretos de serviços e chamadas HTTP com TestClient. A última execução registrada nesta sessão, após o ajuste do formulário administrativo, passou em **137 cenários**. Esse total inclui casos parametrizados; não é uma porcentagem de cobertura.

Os cenários verificam cadastro e confirmação, credenciais, renovação e reutilização, logout, propriedade de sessões, recuperação de senha, papéis/status, tokens inválidos, limites, perfis, documentos, notificações, fila de e-mail, documentação e cabeçalhos.

```
cd backend
.venv/bin/python -m pytest -q
.venv/bin/python -m pytest -v -k "password"
.venv/bin/python -m pytest -x --tb=short
```

Os testes usam SQLite em memória, diretórios temporários e remetentes simulados. Não precisam do servidor HTTP rodando e não devem ser confundidos com testes sobre os dados reais do seu ambiente.

O que a suíte não demonstra sozinha

Não comprova toda disputa concorrente no PostgreSQL, entrega real de e-mails, comportamento de todos os navegadores, carga de produção ou ausência de vulnerabilidades. Também não foi apresentada uma medição de cobertura de linhas/ramificações. Os testes Selenium existentes exercitam principalmente a demonstração; a integração real merece uma suíte própria.

Sintoma	O que verificar primeiro
Falha de conexão com o banco	Credenciais em DATABASE_URL, serviço PostgreSQL e permissões.
Tabela não encontrada	Migrações: executar alembic upgrade head no ambiente correto.
403 no formulário administrativo	detail da resposta, Origin, host utilizado e versão atual da API.
Cadastro/recuperação retorna 503	Configuração do provedor de e-mail e chave de criptografia da fila.
E-mail não chega	Worker em execução; status, attempts e next_attempt_at da outbox.
401 depois de um tempo	Expiração, cookies, origem do refresh e revogação da sessão.

Para testar pelo frontend, inicie API, worker e frontend; cadastre, confirme, entre, recarregue, saia e redefina a senha. Na aba Network, confira as respostas da API. Use contas separadas para verificar permissões e dados privados.

Base no código: [C15] test_authentication.py; backend/README.md; frontend/tests/e2e; verificações da sessão de trabalho

20. Glossário e referências

Mapa para continuar a leitura diretamente no código.

Autenticação: validação da identidade. **Autorização:** decisão de permissão. **Claim:** campo declarado no JWT. **Bearer:** credencial apresentada por quem possui o token. **RBAC:** controle de acesso por papéis. **Hash:** resumo de mão única usado para verificação. **Cifragem:** transformação reversível com uma chave. **Outbox:** fila transacional persistida. **Revogação:** marca que encerra a aceitação de uma sessão ou token. **Origem:** combinação de esquema, host e porta.

Código-fonte consultado

[C1] `backend/app/database.py` e `dbbase.py`: engine, sessões SQLAlchemy e base declarativa.

[C2] `backend/app/main.py`: create_router, endpoints, middleware, CORS e Swagger.

[C3] `services/identity_access/services/identity_service.py`: identidade, credenciais e permissões.

[C4] `services/identity_access/services/session_service.py`: sessões, emissão, refresh e revogação.

[C5] `services/identity_access/models.py`: entidades e restrições de persistência.

[C6] `services/identity_access/schemas.py`: contratos de entrada e saída.

[C7] `services/identity_access/api.py`: dependências, origem, limites e tradução de erros.

[C8] `services/identity_access/security.py`: hashes, segredos, assinatura e validação JWT.

[C9] `services/identity_access/config.py`: padrões e variáveis de ambiente.

[C10] `services/identity_access/services/{profile,media,teacher_application}_service.py`.

[C11] `frontend/src/lib/apiClient.ts` e `components/access/ApiSessionSync.tsx`.

[C12] `services/identity_access/services/email_outbox_service.py` e `email_worker.py`.

[C13] `services/identity_access/services/notification_service.py`: avisos, SMTP e Brevo.

[C14] `services/identity_access/services/rate_limit_service.py`: limites persistentes.

[C15] `backend/tests/test_authentication.py`: cenários automatizados.

Os caminhos services/identity_access/ são relativos a backend/app/. O caminho components/access/ é relativo a frontend/src/.

Referências conceituais oficiais

[R1] RFC 7519 — JSON Web Token (JWT): formato e terminologia das claims.

[R2] RFC 8725 — JSON Web Token Best Current Practices: validação e separação de usos.

[R3] RFC 8037 — Ed25519 em JOSE. O registro da RFC informa atualização pela RFC 9864. O guia descreve o identificador EdDSA usado no código, sem propor uma migração de algoritmo.